

Java 8+ Features for Beginners (Tier 3)

A continuation of the Tier 1 and Tier 2 guides. By now you know lambdas, streams, records, switch expressions, and the new date/time API.

In Tier 3, we cover the more advanced features. These come up in senior-level interviews and are common in modern Java codebases (Java 17 and Java 21).

What's New in Tier 3?

1. Pattern matching for instanceof (Java 16)
2. Pattern matching for switch (Java 21)
3. Record patterns (Java 21)
4. CompletableFuture for async work (Java 8)
5. Virtual threads (Java 21)
6. The new HTTP client (Java 11)
7. Sequenced collections (Java 21)

Some of these are very new, but they are showing up in interviews quickly. Let us go through them one by one.

1. Pattern Matching for instanceof (Java 16)

This is a small change that makes a big difference in readability.

The old way

You had to check the type, then cast separately.

```
java
Object obj = "Hello";

if (obj instanceof String) {
    String s = (String) obj;
    System.out.println(s.length());
}
```

That is two lines of code for what is really one idea: "if it is a String, use it as a String".

The new way

```
java
```

```
Object obj = "Hello";

if (obj instanceof String s) {
    System.out.println(s.length());
}
```

The variable `s` is declared and assigned automatically if the check succeeds. No casting needed.

Combining checks

```
java

if (obj instanceof String s && s.length() > 5) {
    System.out.println("Long string: " + s);
}
```

You can use the variable in the same condition.

Negation

```
java

if (!(obj instanceof String s)) {
    return;
}

// Here, s is in scope and is a String
System.out.println(s.length());
```

This is a clean way to handle the "early return" pattern.

Real example: equals method

The old way:

```
java

@Override
public boolean equals(Object o) {
    if (!(o instanceof Person)) return false;
    Person p = (Person) o;
    return name.equals(p.name) && age == p.age;
}
```

The new way:

```
java
```

```
@Override
public boolean equals(Object o) {
    return o instanceof Person p
        && name.equals(p.name)
        && age == p.age;
}
```

Cleaner, less code, less chance of bugs.

2. Pattern Matching for switch (Java 21)

In Tier 2 we saw switch expressions. Now switch can also match on types, just like `instanceof`.

The basic idea

```
java

Object obj = "Hello";

String result = switch (obj) {
    case Integer i -> "Number: " + i;
    case String s -> "String of length " + s.length();
    case List<?> list -> "List with " + list.size() + " items";
    case null -> "It is null";
    default -> "Unknown type";
};
```

Each case checks the type and gives you a typed variable. Notice you can also handle `null` as a case, which the old switch could not do.

Why this is powerful

It replaces long chains of `if-else if` blocks.

The old way:

```
java
```

```
String result;
if (obj instanceof Integer i) {
    result = "Number: " + i;
} else if (obj instanceof String s) {
    result = "String: " + s;
} else if (obj instanceof List<?> list) {
    result = "List of " + list.size();
} else {
    result = "Unknown";
}
```

The new way is cleaner and easier to read.

Guarded patterns with `when`

You can add an extra condition.

```
java
String describe(Object obj) {
    return switch (obj) {
        case Integer i when i > 0 -> "positive number: " + i;
        case Integer i when i < 0 -> "negative number: " + i;
        case Integer i -> "zero";
        case String s when s.isBlank() -> "blank string";
        case String s -> "string: " + s;
        default -> "something else";
    };
}
```

The order of cases matters. Earlier cases are checked first.

Combining with sealed classes

This is where the magic happens. With sealed classes, the compiler knows all possible types and checks that you handle every one.

```
java
```



```
sealed interface Shape permits Circle, Square, Triangle {}

record Circle(double radius) implements Shape {}
record Square(double side) implements Shape {}
record Triangle(double base, double height) implements Shape {}

double area(Shape s) {
    return switch (s) {
        case Circle c -> Math.PI * c.radius() * c.radius();
        case Square sq -> sq.side() * sq.side();
        case Triangle t -> 0.5 * t.base() * t.height();
        // No default needed, all cases covered
    };
}
```

If you later add `Hexagon` to the sealed interface, the compiler tells you to update this switch. This catches bugs at compile time.

3. Record Patterns (Java 21)

Record patterns let you "unpack" a record's fields directly in the pattern.

The basic idea

```
java

record Point(int x, int y) {}

Object obj = new Point(3, 4);

if (obj instanceof Point(int x, int y)) {
    System.out.println("x = " + x + ", y = " + y);
}
```

In one line, you check the type AND get all the fields out as variables. No need to call `point.x()` and `point.y()` separately.

In switch

```
java
```

```
String describe(Object obj) {
    return switch (obj) {
        case Point(int x, int y) -> "Point at " + x + ", " + y;
        case String s -> "String: " + s;
        default -> "Other";
    };
}
```

Nested patterns

You can unpack records inside records.

```
java

record Point(int x, int y) {}
record Line(Point start, Point end) {}

Line line = new Line(new Point(0, 0), new Point(3, 4));

if (line instanceof Line(Point(int x1, int y1), Point(int x2, int y2))) {
    double distance = Math.sqrt(Math.pow(x2 - x1, 2) + Math.pow(y2 - y1, 2));
    System.out.println("Length: " + distance);
}
```

This is very clean compared to the old way of calling `line.start().x()` and so on.

Combined with sealed classes and switch

This is the modern Java sweet spot.

```
java

sealed interface Shape permits Circle, Rectangle {}

record Circle(double radius) implements Shape {}
record Rectangle(double width, double height) implements Shape {}

double area(Shape s) {
    return switch (s) {
        case Circle(double r) -> Math.PI * r * r;
        case Rectangle(double w, double h) -> w * h;
    };
}
```

You match on the type and extract the fields all in one step.

Using `var` in patterns

If the types are obvious, use `var`.

```
java
case Point(var x, var y) -> "Point: " + x + ", " + y;
```

Same as writing `int x, int y` but a bit shorter.

4. `CompletableFuture` : Async Programming

`CompletableFuture` lets you run code in the background and chain steps together. It was added in Java 8 and is still the main way to do async work in Java (until virtual threads, which we cover next).

The basic idea

```
java
CompletableFuture<String> future = CompletableFuture.supplyAsync(() -> {
    // This runs in another thread
    Thread.sleep(1000);
    return "Hello";
});

System.out.println("Doing other work");

String result = future.get(); // waits for the result
System.out.println(result);
```

`supplyAsync` runs the work in a background thread. The main thread keeps going. When you call `get()`, it waits until the work is done.

Chaining steps with `thenApply`

`thenApply` runs more work after the previous step finishes. It is like `map` for futures.

```
java
CompletableFuture<String> future = CompletableFuture
    .supplyAsync(() -> fetchUserName())
    .thenApply(name -> name.toUpperCase())
    .thenApply(name -> "Hello, " + name);

String result = future.get(); // "Hello, ALICE"
```

thenAccept : just consume the result

If you do not need to return anything from the next step.

```
java

CompletableFuture
    .supplyAsync(() -> fetchData())
    .thenAccept(data -> System.out.println("Got: " + data));
```

thenRun : run after, ignore the result

```
java

CompletableFuture
    .supplyAsync(() -> doSomething())
    .thenRun(() -> System.out.println("Done"));
```

Handling errors with exceptionally

```
java

CompletableFuture<String> future = CompletableFuture
    .supplyAsync(() -> {
        if (Math.random() > 0.5) {
            throw new RuntimeException("Oops");
        }
        return "Success";
    })
    .exceptionally(ex -> {
        System.err.println("Error: " + ex.getMessage());
        return "Default Value";
    });
```

`exceptionally` is like `catch` for futures. It runs only if there was an error.

handle : do both at once

```
java
```

```
CompletableFuture<String> future = CompletableFuture
    .supplyAsync(() -> riskyWork())
    .handle((result, error) -> {
        if (error != null) {
            return "Failed: " + error.getMessage();
        }
        return "Got: " + result;
    });
```

Combining two futures

java

```
CompletableFuture<String> name = CompletableFuture.supplyAsync(() -> "Alice");
CompletableFuture<Integer> age = CompletableFuture.supplyAsync(() -> 25);

CompletableFuture<String> combined = name.thenCombine(age,
    (n, a) -> n + " is " + a + " years old");

System.out.println(combined.get()); // "Alice is 25 years old"
```

`thenCombine` waits for both futures and combines their results.

Waiting for many futures

java

```
CompletableFuture<String> f1 = CompletableFuture.supplyAsync(() -> "A");
CompletableFuture<String> f2 = CompletableFuture.supplyAsync(() -> "B");
CompletableFuture<String> f3 = CompletableFuture.supplyAsync(() -> "C");

// Wait for all to finish
CompletableFuture.allOf(f1, f2, f3).join();

// Or get the first one to finish
CompletableFuture<Object> first = CompletableFuture.anyOf(f1, f2, f3);
```

get vs join

- `get()` throws checked exceptions, so you have to handle them.
- `join()` throws unchecked exceptions, simpler in lambdas.

java

```
String r1 = future.get();    // checked
String r2 = future.join();   // unchecked
```

Most code prefers `join()` because it is cleaner.

Common pattern: parallel API calls

```
java

CompletableFuture<User> userFuture = CompletableFuture
    .supplyAsync(() -> userService.fetch(userId));

CompletableFuture<List<Order>> ordersFuture = CompletableFuture
    .supplyAsync(() -> orderService.fetch(userId));

CompletableFuture<UserProfile> profile = userFuture.thenCombine(
    ordersFuture,
    (user, orders) -> new UserProfile(user, orders)
);
```

Both calls happen in parallel. The combine waits for both.

5. Virtual Threads (Java 21)

Virtual threads are one of the biggest changes in Java in the last decade. They are lightweight threads. You can have millions of them at once.

The problem they solve

Regular Java threads are mapped to OS threads. Each one uses about 1 MB of memory. So if your server handles 10,000 connections, you cannot just create 10,000 threads.

Before virtual threads, the answer was async code (like `CompletableFuture` or reactive programming). That works but it is hard to write and read.

Virtual threads let you write simple, blocking code that scales to massive concurrency.

Basic usage

```
java
```

```
// Create and start a virtual thread
Thread vt = Thread.startVirtualThread(() -> {
    System.out.println("Running in: " + Thread.currentThread());
});

vt.join();
```

Using an executor

This is the more common pattern.

```
java

try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 10_000; i++) {
        int taskId = i;
        executor.submit(() -> {
            System.out.println("Task " + taskId);
            // Simulate some I/O
            Thread.sleep(1000);
            return null;
        });
    }
}
```

You can submit thousands of tasks. Each one gets its own virtual thread. This would crash with regular threads.

Why this matters

You can write code like this:

```
java

public String fetchUserData(int userId) {
    User user = userService.fetch(userId);           // blocks
    List<Order> orders = orderService.fetch(userId); // blocks
    return formatProfile(user, orders);
}
```

It looks simple and blocks at each step. With regular threads, this would not scale. With virtual threads, you can handle thousands of concurrent calls without complex async code.

When virtual threads help

- I/O-bound work (HTTP calls, database queries, file reads)
- High concurrency servers (web apps handling many users)

- When you want simple blocking code that scales

When virtual threads do NOT help

- CPU-bound work. Virtual threads do not give you more CPU. Use regular threads or parallel streams for heavy computation.
- Code that holds onto OS resources for a long time without yielding.

Virtual threads vs CompletableFuture

For most server code, virtual threads are now the simpler choice. Write blocking code, run it on virtual threads, and you get massive concurrency for free.

But CompletableFuture is still useful for orchestrating multiple async operations, especially when combining results.

Things to know

1. Virtual threads are managed by the JVM, not the OS.
 2. They are cheap to create. Do not pool them. Create a new one per task.
 3. They work with most existing code that uses `Thread`, `synchronized`, or `BlockingQueue`.
 4. The `synchronized` keyword can sometimes pin a virtual thread (block it from being unmounted). The newer `ReentrantLock` does not have this issue. This is an active area of improvement in Java.
-

6. HTTP Client (Java 11)

A modern, built-in HTTP client. Replaces the old `URLConnection` which was painful to use.

The basics

You need three things:

1. An `HttpClient` (you can reuse this)
2. An `HttpRequest`
3. A way to handle the response

A simple GET request

```
java
```



```
HttpClient client = HttpClient.newHttpClient();

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com/users"))
    .GET()
    .build();

HttpResponse<String> response = client.send(
    request,
    HttpResponse.BodyHandlers.ofString()
);

System.out.println("Status: " + response.statusCode());
System.out.println("Body: " + response.body());
```

`BodyHandlers.ofString()` says "give me the body as a String". There are other handlers like `ofByteArray()`, `ofFile(path)`, and `ofInputStream()`.

Adding headers

```
java

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com/users"))
    .header("Authorization", "Bearer my-token")
    .header("Accept", "application/json")
    .GET()
    .build();
```

POST request

```
java
```

```
String json = """
    {
        "name": "Alice",
        "age": 25
    }
    """;

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com/users"))
    .header("Content-Type", "application/json")
    .POST(HttpRequest.BodyPublishers.ofString(json))
    .build();

HttpResponse<String> response = client.send(
    request,
    HttpResponse.BodyHandlers.ofString()
);
```

Async request

```
java

client.sendAsync(request, HttpResponse.BodyHandlers.ofString())
    .thenApply(HttpResponse::body)
    .thenAccept(System.out::println)
    .exceptionally(ex -> {
        System.err.println("Failed: " + ex.getMessage());
        return null;
    });
```

The async version returns a `CompletableFuture`, which you already know how to use.

Setting timeouts

```
java

HttpClient client = HttpClient.newBuilder()
    .connectTimeout(Duration.ofSeconds(10))
    .build();

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com/data"))
    .timeout(Duration.ofSeconds(30))
    .build();
```

Handling JSON

The HTTP client gives you the body as a string. You usually parse it with a library like Jackson or Gson.

```
java

String json = response.body();

ObjectMapper mapper = new ObjectMapper();
User user = mapper.readValue(json, User.class);
```

Why use the new HTTP client

- Built into the JDK, no external library
 - Supports HTTP/2 out of the box
 - Has both sync and async APIs
 - Cleaner than the old `HttpURLConnection`
-

7. Sequenced Collections (Java 21)

A new set of interfaces for collections that have a defined order. They give you nice methods like `getFirst()`, `getLast()`, and `reversed()`.

The new interfaces

- `SequencedCollection<E>`: for ordered collections (like `List`)
- `SequencedSet<E>`: for ordered sets (like `LinkedHashSet`)
- `SequencedMap<K, V>`: for ordered maps (like `LinkedHashMap`)

New methods on List

```
java
```

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3, 4, 5));

list.getFirst();    // 1
list.getLast();     // 5

list.addFirst(0);    // [0, 1, 2, 3, 4, 5]
list.addLast(6);     // [0, 1, 2, 3, 4, 5, 6]

list.removeFirst();  // returns 0
list.removeLast();   // returns 6

List<Integer> reversed = list.reversed();    // [5, 4, 3, 2, 1]
```

The reversed view is "live". Changes to the original affect it.

New methods on LinkedHashSet

Sets did not have a clean way to get the first or last element before. Now they do.

```
java

LinkedHashSet<String> set = new LinkedHashSet<>();
set.add("apple");
set.add("banana");
set.add("cherry");

set.getFirst();    // "apple"
set.getLast();     // "cherry"
set.reversed();    // [cherry, banana, apple]
```

New methods on LinkedHashMap

```
java

LinkedHashMap<String, Integer> map = new LinkedHashMap<>();
map.put("one", 1);
map.put("two", 2);
map.put("three", 3);

map.firstEntry();   // one=1
map.lastEntry();    // three=3

map.pollFirstEntry(); // removes and returns one=1
map.pollLastEntry();  // removes and returns three=3
```

Why this is useful

Before Java 21, getting the first or last element of a list was awkward.

```
java

// Old way (still works)
list.get(0);
list.get(list.size() - 1);

// New way (cleaner)
list.getFirst();
list.getLast();
```

For sets and maps, it was even worse. There was no clean way at all without converting to a list first.

Putting It All Together

Real examples that combine these features.

Example 1: A type-safe message processor

```
java

sealed interface Message permits TextMessage, ImageMessage, VideoMessage {}

record TextMessage(String content) implements Message {}
record ImageMessage(String url, int width, int height) implements Message {}
record VideoMessage(String url, int durationSeconds) implements Message {}

String describe(Message msg) {
    return switch (msg) {
        case TextMessage(String content) -> "Text: " + content;
        case ImageMessage(String url, int w, int h) ->
            "Image " + w + "x" + h + " at " + url;
        case VideoMessage(String url, int seconds) when seconds > 60 ->
            "Long video at " + url;
        case VideoMessage(String url, int seconds) ->
            "Short video (" + seconds + "s) at " + url;
    };
}
```

Example 2: Parallel API calls with CompletableFuture

```
java
```

```
CompletableFuture<User> userFuture = CompletableFuture
    .supplyAsync(() -> userService.fetch(userId));

CompletableFuture<List<Order>> ordersFuture = CompletableFuture
    .supplyAsync(() -> orderService.fetchByUser(userId));

CompletableFuture<List<Address>> addressFuture = CompletableFuture
    .supplyAsync(() -> addressService.fetchByUser(userId));

CompletableFuture.allOf(userFuture, ordersFuture, addressFuture).join();

UserProfile profile = new UserProfile(
    userFuture.join(),
    ordersFuture.join(),
    addressFuture.join()
);
```

All three calls run in parallel. You wait for all to finish, then build the result.

Example 3: Web scraper using HTTP client and virtual threads

```
java
```

```

List<String> urls = List.of(
    "https://example.com/1",
    "https://example.com/2",
    "https://example.com/3"
    // ... thousands more
);

HttpClient client = HttpClient.newHttpClient();

try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    List<Future<String>> futures = urls.stream()
        .map(url -> executor.submit(() -> {
            HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .build();
            HttpResponse<String> res = client.send(
                req, HttpResponse.BodyHandlers.ofString()
            );
            return res.body();
        }))
        .toList();

    futures.forEach(f -> {
        try {
            System.out.println(f.get().substring(0, 100));
        } catch (Exception e) {
            System.err.println("Failed: " + e.getMessage());
        }
    });
}

```

You can scrape thousands of URLs with simple blocking code. Virtual threads handle the concurrency.

Example 4: Find first and last in a list

```

java

```

```
List<Person> people = List.of(
    new Person("Alice", 25),
    new Person("Bob", 30),
    new Person("Carol", 28)
);

Person firstPerson = people.getFirst();
Person lastPerson = people.getLast();

List<Person> reversed = people.reversed();
```

Example 5: Pattern matching for cleaner equals

```
java

record Point(int x, int y) {
    @Override
    public boolean equals(Object o) {
        return o instanceof Point(int ox, int oy)
            && x == ox
            && y == oy;
    }
}
```

(Although records give you `equals` for free, this shows the pattern for non-record classes.)

Example 6: Async pipeline with error handling

```
java

CompletableFuture<String> result = CompletableFuture
    .supplyAsync(() -> fetchRawData())
    .thenApply(data -> parseData(data))
    .thenApply(parsed -> transformData(parsed))
    .thenApply(transformed -> saveResult(transformed))
    .exceptionally(ex -> {
        log.error("Pipeline failed", ex);
        return "FAILED";
    });

System.out.println(result.join());
```

Each step runs after the previous one finishes. Any error short-circuits to the `exceptionally` handler.

Common Mistakes

Mistake 1: Forgetting that pattern variables are scoped

```
java

if (obj instanceof String s) {
    System.out.println(s);    // OK
}
System.out.println(s);    // ERROR: s is not in scope
```

The pattern variable only exists where the check is true.

Mistake 2: Wrong order in switch with guarded patterns

```
java

return switch (obj) {
    case Integer i -> "any int";
    case Integer i when i > 0 -> "positive";    // never reached!
    default -> "other";
};
```

The unguarded case `Integer i` matches first. Always put more specific cases first.

Mistake 3: Calling `get()` on an unfinished `CompletableFuture` without timeout

```
java

String result = future.get();    // could block forever
```

Better:

```
java

String result = future.get(5, TimeUnit.SECONDS);
```

Mistake 4: Pooling virtual threads

```
java

// Wrong
ExecutorService pool = Executors.newFixedThreadPool(100);    // not virtual

// Right
ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor();
```

Do not pool virtual threads. Create a new one per task. They are cheap.

Mistake 5: Using virtual threads for CPU-heavy work

```
java

try (var exec = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 10000; i++) {
        exec.submit(() -> heavyMathCalculation());
    }
}
```

Virtual threads do not give you more CPU. For heavy computation, use a regular fixed thread pool sized to your CPU count.

Mistake 6: Forgetting to close the HttpClient or executor

```
java

// Better: use try-with-resources
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    // submit tasks
}
// executor is closed automatically and waits for all tasks to finish
```

Mistake 7: Mixing blocking and async in CompletableFuture

```
java

// Bad: calls get() inside thenApply, blocking a thread
future.thenApply(data -> {
    return otherFuture.get(); // do not do this
});

// Good: use thenCompose for chaining futures
future.thenCompose(data -> otherFuture);
```

`thenCompose` is like `flatMap` for futures.

Mistake 8: Not handling exceptions in async pipelines

```
java

CompletableFuture.supplyAsync(() -> riskyWork())
    .thenApply(this::process);
// If riskyWork throws, you may never know
```

Always end with `exceptionally` or `handle` to log or recover from errors.

Mistake 9: Trying to mutate a reversed view

```
java  
  
List<Integer> list = new ArrayList<>(List.of(1, 2, 3));  
List<Integer> reversed = list.reversed();  
reversed.add(4);    // changes the original list!
```

The reversed view is connected to the original. Make a copy if you need an independent list.

Quick Cheat Sheet

You want to...	Use
Type check and cast in one step	<code>instanceof Type t</code>
Type-based switch	Pattern matching for switch
Add a condition to a switch case	<code>case Type t when condition</code>
Unpack a record's fields	Record pattern: <code>Point(int x, int y)</code>
Run work in another thread	<code>CompletableFuture.supplyAsync(...)</code>
Chain steps after async work	<code>.thenApply(...)</code> , <code>.thenAccept(...)</code>
Handle errors in async	<code>.exceptionally(...)</code> or <code>.handle(...)</code>
Combine two futures	<code>.thenCombine(other, (a, b) -> ...)</code>
Wait for many futures	<code>CompletableFuture.allOf(...)</code>
Lightweight thread for I/O work	Virtual thread
Run many tasks with virtual threads	<code>Executors.newVirtualThreadPerTaskExecutor()</code>
Make an HTTP GET call	<code>HttpClient.send(request, BodyHandlers.ofString())</code>
Async HTTP call	<code>client.sendAsync(...)</code>
First or last item in a List	<code>list.getFirst()</code> , <code>list.getLast()</code>
Reversed view of a List	<code>list.reversed()</code>
First or last entry in a LinkedHashMap	<code>map.firstEntry()</code> , <code>map.lastEntry()</code>

Final Tips

1. Pattern matching is the new normal. Use `instanceof` with a pattern variable everywhere you used to cast manually.
2. The combo of sealed interfaces, records, and pattern matching is one of the cleanest ways to model data in modern Java.
3. `CompletableFuture` is still useful, but for new server code, virtual threads make blocking code simple and fast. Pick the right tool for the job.
4. Virtual threads are for I/O work. Regular threads or parallel streams are for CPU work.
5. The new HTTP client is good enough for most needs. You do not need a separate library for basic HTTP work.
6. For sequenced collections, the new methods are small but very nice for readability. Use them.

You now know the most important Java 8 through Java 21 features. You have a complete picture of modern Java for interviews:

- 3 guides on streams
- 3 guides on other Java 8+ features

This is a strong foundation. The rest is practice.

If you want to keep going, good next topics are:

- Java Collections deep dive (when to use which)
- Concurrency and multithreading basics
- Exception handling patterns
- Common design patterns
- JVM internals and performance basics

Happy coding!